A Mini Project Synopsis on

SmartClaim AI: Intelligent Vehicle Damage Detection & Insurance Claim Estimator

B.E. - I.T Engineering

Submitted By

Sunny Gupta	**23104136**
Pranav Ghodke	**23104115**
Aditya Chavan	**23104090**

Under The Guidance Of

Ms. Snehal Mali



DEPARTMENT OF INFORMATION TECHNOLOGY

A.P. SHAH INSTITUTE OF TECHNOLOGY

G.B. Road, Kasarvadavali, Thane (W), Mumbai-400615

UNIVERSITY OF MUMBAI

Academic year: 2026-27

CERTIFICATE

This is to certify that the Mini Project report on **SmartClaim AI : Intelligent Vehicle Damage Detection & Insurance Claim Estimator** has been submitted by Sunny Gupta (23104136), Pranav Ghodke (23104115) and Aditya Chavan (23104090) who are a Bonafide students of A. P. Shah Institute of Technology, Thane, Mumbai, as a partial fulfilment of the requirement for the degree in **Information Technology**, during the academic year **2026-2027** in the satisfactory manner as per the curriculum laid down by University of Mumbai.

Ms. Snehal Mali

Guide

External Examiner(s)

1.

Place: A.P. Shah Institute of Technology, Thane

Date:

ACKNOWLEDGEMENT

This project would not have come to fruition without the invaluable help of our guide **Ms. Snehal Mali**. Expressing gratitude towards our HoD, **Dr. Kiran Deshpande**, and the Department of Information Technology for providing us with the opportunity as well as the support required to pursue this project.

ABSTRACT

Motor insurance claims in India still depend on a surveyor visiting the vehicle, inspecting the damage and preparing a repair estimate by hand. The process is slow, varies from surveyor to surveyor, and gives the vehicle owner little idea of what the claim is worth until days later. **SmartClaim AI** is an AI assistant that turns a single photograph of a damaged vehicle, together with a few vehicle and policy details, into an instant, itemised repair and claim estimate.

The system combines three deep-learning models with a transparent pricing engine. A YOLOv8 classification model identifies where the damage is (front or rear) and how severe it is (normal, breakage, crushed), reaching **80.35% validation accuracy** and outperforming a custom CNN, EfficientNet-B0 and ResNet50 trained on the same 2,300-image dataset. Two YOLO11 instance-segmentation models then locate 21 car parts (mask mAP@0.5 of **0.81**) and eight damage types; each damage region is attributed to the part it overlaps, so the estimate names the actual parts to repair or replace.

A rule-based estimator prices every part using segment-wise costs, city-tier labour rates, paint and GST, and then applies Indian motor-insurance rules — the IRDAI depreciation schedule, compulsory and voluntary deductibles, the zero-depreciation add-on and the 75% constructive total-loss threshold — to split the bill into the amount the insurer pays and the amount the customer pays. An analysis of a public claims dataset showed that tabular claim records cannot predict repair cost from damage information, which motivated this part-level design. The complete pipeline runs on an Apple Silicon GPU and is delivered through a web application that returns the annotated photo and a low-to-high estimate in about two seconds.

Keywords: vehicle damage detection, insurance claim estimation, YOLO, instance segmentation, transfer learning, IRDAI depreciation, computer vision

TABLE OF CONTENTS

1. Introduction....1
1.1 Purpose....1
1.2 Objectives....1
1.3 Scope....2
2. Literature Survey....3
3. Problem Definition....4
4. Proposed System....5
4.1 System Architecture....5
4.2 Workflow....6
4.3 Algorithm....7
5. Technology Stack....10
5.1 Why YOLO Models....10
5.2 Why a Rule-Based Estimator....10
6. Implementation....11
6.1 Datasets....11
6.2 Development Environment and GPU Acceleration....12
6.3 Model Training....12
6.4 Damage-to-Part Attribution....13
6.5 Pricing Engine and Policy Rules....13
6.6 Web Application....14
6.7 Testing....14
7. Results....15
7.1 Location and Severity Classification....15
7.2 Part and Damage Segmentation....16
7.3 End-to-End Claim Estimates....17
7.4 Performance....18
8. Conclusion and Future Scope....19
8.1 Future Scope....19
References....20

1. Introduction

1.1 Purpose

After a road accident, the first question a vehicle owner asks is how much the repair will cost and how much of it the insurance company will pay. Today that answer comes only after a claim is registered, a surveyor is assigned, the vehicle is inspected in person and a workshop prepares an estimate. Each step adds days, and the result depends heavily on the experience of the individual surveyor. Insurers, in turn, spend a large share of their claim-handling cost on inspecting small claims that could be assessed from photographs.

The purpose of **SmartClaim AI** is to show that this first assessment can be automated end to end. The user uploads a photograph of the damaged vehicle and enters a few details that every policy document already contains — the type of car, its age, engine capacity, Insured Declared Value (IDV), city and whether the policy has a zero-depreciation add-on. Within about two seconds the system returns:

- an annotated photograph showing the detected car parts and damage regions,
- the location and severity of the damage with the model's confidence,
- an itemised repair plan listing each part and whether it should be repaired, repainted or replaced,
- a low-to-high repair estimate in rupees including labour, paint and GST, and
- the claim settlement: how much the insurer pays after depreciation and deductible, and how much the customer pays.

Every rupee in the estimate can be traced back to a detected part and a documented rule.

1.2 Objectives

1. To build and compare deep-learning models that classify damage location (front/rear) and severity (normal, breakage, crushed).
2. To segment which car part is damaged and what type of damage it has.
3. To attribute each damage region to the car part it lies on.

4. To design a transparent pricing engine using part prices, car segment, labour rates, paint and GST.
5. To apply IRDAI rules: depreciation, deductibles, zero-dep cover and total loss.
6. To deliver the complete pipeline as an easy-to-use web application.

1.3 Scope

The scope of SmartClaim AI is as follows:

- Can be used to assess damage to passenger cars (hatchback, sedan, SUV and luxury segments) from a single front or rear photograph, classifying it into six location and severity classes: front and rear × breakage, crushed and normal.
- Can be used to identify 21 exterior car parts (bumpers, bonnet, boot lid, doors, fenders, lamps, glass, mirrors, wheels, licence plate) and eight damage types (dent, scratch, broken part, missing part, crack, paint chip, flaking, corrosion), and to link each damage to the part it lies on.
- Can be used to generate an itemised repair estimate from a catalogue of 23 priced parts with segment-wise prices and city-tier labour rates, and to compute the claim settlement as per the IRDAI depreciation schedule, deductibles, zero-depreciation cover and the constructive total-loss rule.
- Can be used by vehicle owners and claim handlers through a browser-based web application to get an instant first estimate, and can be extended with multi-photo assessment, live part prices, fraud screening and a mobile application.

The estimate produced by SmartClaim AI is an indicative first assessment that helps the customer and the insurer start a claim with a shared, explainable number; the final settlement remains subject to the insurer's survey as required by regulation.

2. Literature Survey

The research papers [1]–[7] summarised in Table 2.1 shaped the design choices of SmartClaim AI.

Table 2.1: Literature survey summary

Sr. No.	Title	Authors	Year	Findings and relevance to SmartClaim AI
1	An Insurtech Platform to Support Claim Management Through the Automatic Detection and Estimation of Car Damage from Pictures (Electronics) [1]	M. M. H. Atanasious, V. Becchetti, A. Giuseppi et al.	2024	Presented Insoore AI, an Italian insurtech platform that uses computer vision to automate damage identification and repair-cost estimation for claims.
2	YOLOv11: An Overview of the Key Architectural Enhancements (arXiv) [2]	R. Khanam, M. Hussain	2024	Reviews YOLO11 (C3k2 and C2PSA blocks) and its instance segmentation support. YOLO11n-seg powers our part segmenter (mask mAP50 0.81) and damage segmenter.
3	Automated Vehicle Damage Classification Using the Three-Quarter View Car Damage Dataset and Deep Learning Approaches (Heliyon) [3]	D. Lee, J. Lee, E. Park	2024	Introduced a three-quarter-view car damage dataset labelled by side (front/rear) and damage type, and compared pretrained backbones such as ResNet-50 and EfficientNet-B0. We use the same front/rear classes and a similar backbone comparison.
4	YOLOv8: A Novel Object Detection Algorithm with Enhanced Performance and Robustness (IEEE ADICS) [4]	R. Varghese, S. M.	2024	Describes YOLOv8 and its accuracy and speed gains over earlier YOLO versions. YOLOv8n-cls was the best of our five classifiers (80.35% accuracy).
5	CES: Cost Estimation System for Enhancing the Processing of Car Insurance Claims (JoCC) [5]	A. S. Elbhrawy, M. A. Belal, M. S. Hassanein	2024	Proposed a deep-learning system that recognises damaged body parts and damage level from photos to estimate claim cost (2,508 car photos, mAP 0.66).
6	Segment Anything (ICCV) [6]	A. Kirillov, E. Mintun, N. Ravi et al.	2023	Introduced a promptable segmentation model trained on over one billion masks (SA-1B). Pixel-level masks let SmartClaim AI measure how much of a part a damage covers.
7	CarDD: A New Dataset for Vision-based Car Damage Detection (IEEE T-ITS) [7]	X. Wang, W. Li, Z. Wu	2023	Released 4,000 high-resolution images with over 9,000 labelled instances of six damage types, establishing fine-grained car damage segmentation as a practical task.

Gap identified. Most published work stops at classifying or segmenting damage [2]–[4], [6], [7]. Systems that estimate cost [1], [5] give no itemised, explainable breakdown and do not apply the IRDAI depreciation and deductible rules that decide what an Indian policyholder is paid. SmartClaim AI closes this gap with part-level segmentation and an explainable, regulation-aware estimator.

3. Problem Definition

Motor own-damage claims follow a manual, inspection-driven process. For the customer the claim amount is unknown until a surveyor visits; for the insurer every claim, however small, needs a person on site. The specific problems addressed by this project are:

1. **Slow first assessment:** the customer waits for an inspection before learning whether a claim is worth filing.
2. **Inconsistent estimates:** two surveyors can value the same damage differently because there is no shared, explainable method.
3. **Opaque settlements:** customers rarely understand why the amount paid is lower than the garage bill — depreciation and deductibles are applied out of sight.
4. **Damage alone does not decide cost:** the same dent costs very different amounts on a hatchback bumper and on a luxury SUV door, so a useful estimate must know the part, the vehicle segment and the policy terms.
5. **Existing claim data cannot be reused for pricing:** public insurance datasets record claim amounts, but not in a form that links them to the visible damage.

The last point was verified experimentally on a public auto-insurance claims dataset (1,000 claims, 40 attributes). Restricted to collisions, a gradient-boosting model predicting the repair claim reached a cross-validated R^2 of -0.14, i.e. worse than predicting the average. Minor damage, major damage and total loss all average about $45,000 in that dataset.

Problem statement. Design and implement a system that takes a photograph of a damaged car and basic vehicle and policy details, identifies the damaged parts and the nature of the damage, and produces an itemised, explainable repair estimate and claim settlement consistent with Indian motor-insurance rules — without relying on historical claim amounts that do not reflect the visible damage.

4. Proposed System

SmartClaim AI is organised as a layered system in which deep-learning models produce visual evidence and a transparent rule engine converts that evidence into money. Keeping the two concerns separate means every model can be retrained without touching the pricing rules, and every price can be changed without retraining a model.

4.1 System Architecture

Figure 4.1 shows the components of the system, organised in five layers, with the offline model-training pipeline alongside:

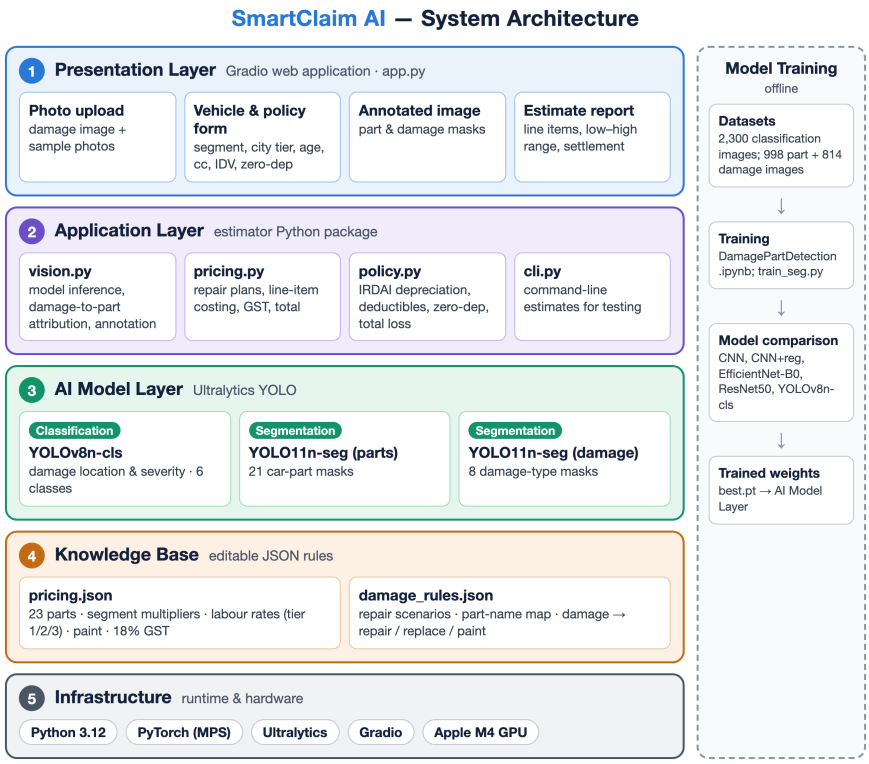


Figure 4.1: System architecture of SmartClaim AI

- **Presentation layer:** the Gradio web application (*app.py*) with a photo upload, a vehicle and policy form, the annotated result image and the estimate report.
- **Application layer:** the *estimator* Python package. *vision.py* runs the models and attributes damage to parts; *pricing.py* builds repair plans and costs each line item; *policy.py* encodes the IRDAI depreciation schedule, deductibles, zero-depreciation cover and the total-loss rule; *cli.py* produces estimates from the command line.

- **AI model layer:** three Ultralytics YOLO models — YOLOv8n-cls for damage location and severity, and two YOLO11n-seg models for 21 car parts and 8 damage types.
- **Knowledge base:** two editable JSON files. *pricing.json* holds the part catalogue, segment multipliers, labour rates, paint cost and GST; *damage_rules.json* holds the repair scenarios, the part-name mapping and the rules that turn a damage type into a repair action.
- **Infrastructure:** Python 3.12, PyTorch with the Metal Performance Shaders (MPS) backend, Ultralytics and Gradio, running on the Apple M4 GPU.
- **Model training (offline):** the datasets are used to train and compare the models in the training notebook and scripts; the resulting weights are loaded by the AI model layer.

4.2 Workflow

Figure 4.2 traces a single request through the system, from the uploaded photo to the settlement shown to the user. The two decision points that matter most are whether the damage model finds any damage (if not, the classifier's assessment is used on its own) and whether the repair bill exceeds 75% of the IDV, in which case the claim is settled as a constructive total loss.

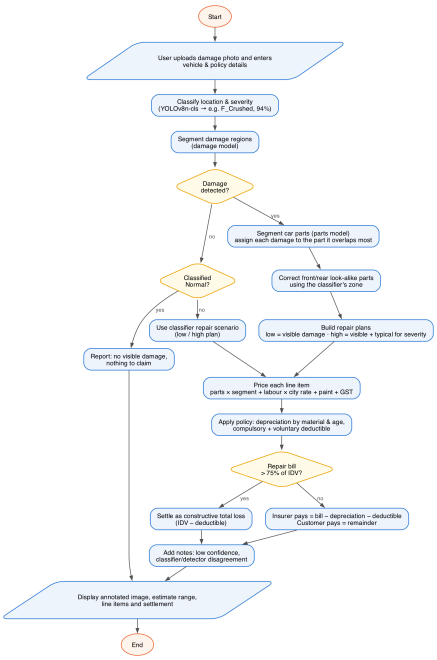


Figure 4.2: End-to-end workflow of a claim estimate

4.3 Algorithm

The system is driven by three algorithms that run in sequence for every photograph.

4.3.1 Algorithm 1: Damage Location and Severity Classification

Input: RGB photograph I. **Output:** label $y \in \{F, R\} \times \{Breakage, Crushed, Normal\}$ and confidence c.

1. Load the YOLOv8n-cls weights once on the GPU (MPS), convert the photo to RGB and resize it to 224 × 224.
2. Pass the image through the network and apply softmax: p(k) = exp(z_k) / Σ_j exp(z_j).
3. Normalise pixel values and pass the tensor through the network to obtain six class scores *z*Select y = argmax_k p(k) with confidence c = max_k p(k), split y into zone and severity, and add a manual-inspection note if c < 0.5.

4.3.2 Algorithm 2: Damage-to-Part Attribution

Input: image I, zone from Algorithm 1. **Output:** list of detections (damage type, part, area fraction, part fraction, confidence).

1. Run the damage segmentation model with confidence threshold 0.3 to obtain damage masks $D_1 \ldots D_n$ and their classes.
2. Run the car-part segmentation model on the same image to obtain part masks $P_1 \ldots P_m$ at the original resolution.
3. For every damage mask D_i compute the overlap with each part: $\text{overlap}(i, j) = |D_i \cap P_j| / |D_i|$.
4. Assign D_i to the part j *with the largest overlap if overlap(i, j)* ≥ 0.2; otherwise fall back to a default part for that damage type in the classified zone (for example a crack in the front zone → front bumper).
5. If the part is a front/rear look-alike (bumper, lamp, bonnet/boot lid, windscreen) and disagrees with the classifier's zone, replace it with its counterpart in the classified zone.
6. Record the area fraction $|D_i| / |I|$ and the part fraction $|D_i| / |P_j^*|$, used later to decide repair versus replacement.

4.3.3 Algorithm 3: Repair Estimate and Claim Settlement

Input: classifier label, detections, vehicle (segment, city tier) and policy (age, engine cc, IDV, zero-dep, voluntary deductible). **Output:** low and high scenarios, each with line items and settlement.

1. **Build plans.** For each detection, look up the [low, high] repair action of its damage type (e.g. dent → [repair, repair], broken part → [replace, replace], scratch → [paint, paint]). A dent covering more than 30% of its part is upgraded to *replace* in the high plan. The **low plan** contains only visible damage; the **high plan** adds the classifier's typical repair scenario for the detected severity, covering damage hidden behind the panels. When no damage is detected the classifier's scenarios are used directly. Duplicate parts keep the most extensive action.
2. **Price each line item** using the part catalogue (s = segment multiplier, r = city labour rate per hour):

$$\textit{parts} = \textit{price}(\textit{part}, \textit{action}) \times s \qquad \textit{labour} = \textit{hours}(\textit{part}, \textit{action}) \times r$$

$$paint = 2{,}800 \times s \quad \textit{(painted panels only)}$$

$$item = (parts + labour + paint) \times 1.18 \qquad \textit{(GST at 18\%)}$$

3. **Depreciation** applies to replaced parts by material — rubber, nylon and plastic 50%, fibreglass 30%, glass 0%, metal 0–50% by vehicle age — and 50% on the material share (25%) of paint. With a zero-depreciation add-on it is zero.
4. **Deductible** = ₹1,000 for engines up to 1,500 cc or ₹2,000 above, plus any voluntary deductible.
5. **Settlement:** if the bill exceeds 75% of the IDV, the vehicle is a constructive total loss and the insurer pays IDV − deductible; otherwise insurer pays = bill − depreciation − deductible. The customer pays the remainder.

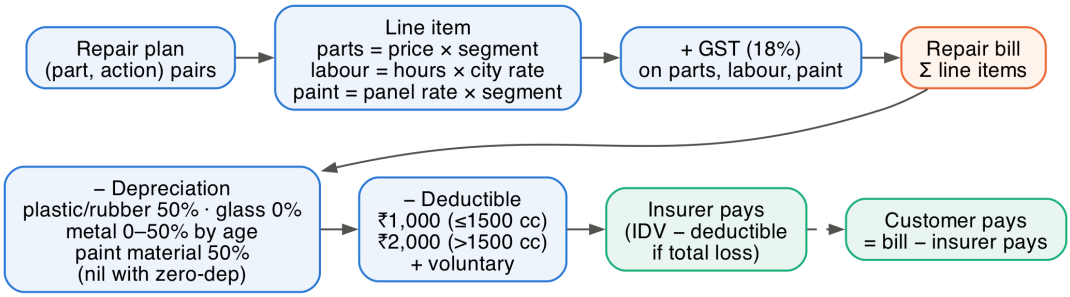


Figure 4.3: From repair plan to settlement (Algorithm 3)

5. Technology Stack

SmartClaim AI is written entirely in Python. Table 5.1 lists the technologies used in each part of the system.

Table 5.1: Technology stack

Layer	Technology	Purpose in SmartClaim AI
Deep learning	PyTorch 2.14, torchvision 0.29	Model definition, transfer learning (ResNet50, EfficientNet-B0) and GPU training
Detection & segmentation	Ultralytics 8.4 (YOLOv8n-cls, YOLO11n-seg)	Training and inference of the three production models; ONNX export
GPU acceleration	Apple Metal Performance Shaders (MPS)	Training and inference on the Apple M4 GPU through PyTorch
Data processing	NumPy, pandas, Pillow	Image loading, RGBA→RGB conversion, dataset statistics, results analysis
Pricing engine	Python dataclasses + JSON	Repair plans, line-item costing, IRDAI depreciation and settlement rules
Web application	Gradio 6.30	Photo upload, vehicle/policy form, annotated image and estimate display
Testing	pytest	32 automated tests covering depreciation, deductibles, plans and settlement
Hardware	Apple M4 (10-core CPU, 10-core GPU), 16 GB unified memory, macOS 26	Development, training and deployment

5.1 Why YOLO Models

YOLO models were chosen for all three vision tasks because they offer pretrained weights, train quickly on small datasets, run in real time and share one API for training, inference and export. The 1.4-million-parameter YOLOv8n-cls was both the smallest and the most accurate classifier, and the nano segmentation models keep the full pipeline at about two seconds per photo.

5.2 Why a Rule-Based Estimator

Repair pricing is governed by part prices, labour rates and published insurance rules, not by patterns a model must discover. Encoding them as JSON data and code makes every figure explainable, lets an insurer update prices without retraining, and avoids claim histories which, as shown in Chapter 3, do not reflect the visible damage.

6. Implementation

6.1 Datasets

6.1.1 Damage Location and Severity Dataset

The classifier was trained on 2,300 photographs of cars sorted into six folders by damage location and severity. Every image was verified to decode correctly; 310 images stored with an alpha channel are converted to RGB during loading. Table 6.1 gives the class distribution and Figure 6.1 shows one example per class.

Table 6.1: Damage location and severity dataset

Class	Meaning	Images
F_Breakage	Front, broken parts	500
F_Crushed	Front, crushed / deformed	400
F_Normal	Front, no damage	500
R_Breakage	Rear, broken parts	300
R_Crushed	Rear, crushed / deformed	300
R_Normal	Rear, no damage	300
Total	**75% training (1,725) / 25% validation (575)**	**2,300**



Figure 6.1: Sample images from the six damage location and severity classes

6.1.2 Car Parts and Damage Segmentation Dataset

Part and damage segmentation use the open *Car Parts and Damages* dataset (MIT licence), which provides polygon annotations in Supervisely format: 998 images labelled with 21 car parts (15,767 polygons) and 814 images labelled with eight damage types (9,084 polygons). A converter written for this project (*prepare_supervisely.py*) normalises

class names, converts every polygon to the YOLO segmentation format and creates a reproducible 85/15 train–validation split.

Table 6.2: Segmentation datasets after conversion

Dataset	Train	Validation	Classes (instances)
Car parts	849	149	21 classes, e.g. back window 1,237 · mirror 987 · front window 951 · fender 935 · front door 917 · headlight 914 · … · back windshield 271
Damage	692	122	scratch 3,242 · dent 1,664 · broken part 1,500 · paint chip 1,356 · missing part 632 · flaking 337 · corrosion 277 · cracked 76

6.2 Development Environment and GPU Acceleration

All training and inference run on an Apple M4 laptop with 16 GB of unified memory. The training code selects the best available device automatically (CUDA, then the Apple MPS GPU, then CPU).

Data loading used four worker processes, since JPEG decoding rather than the GPU was the bottleneck; batch 32 at 224 × 224 (about 1.3 GB of GPU memory) was used for classification and batch 16 at 640 × 640 for segmentation.

6.3 Model Training

6.3.1 Location and Severity Classifiers

Five classifiers were trained on the same split to choose the best architecture. All used 224 × 224 inputs, ImageNet normalisation and augmentation with horizontal flips, occasional vertical flips (p = 0.1), ±15° rotation and colour jitter. Table 6.3 lists their configurations.

Table 6.3: Classifier training configuration

Model	Approach	Optimiser / LR	Epochs	Time
Baseline CNN	3 conv blocks + 512-unit FC	Adam, 1e-3	6	39 s
CNN + Regularisation	Baseline + batch norm, dropout 0.5	Adam, 1e-3, wd 1e-4	5	35 s
EfficientNet-B0	Frozen ImageNet backbone, new head	Adam, 1e-3	5	55 s
ResNet50	ImageNet; layer4 + FC tuned, dropout 0.39	Adam, 8.7e-4	10	275 s
YOLOv8n-cls	Pretrained YOLOv8 nano, batch 32	Ultralytics default	10	3.2 min

6.3.2 Part and Damage Segmentation Models

Both segmentation models start from COCO-pretrained YOLO11n-seg weights and were trained at 640 × 640 with batch 16 on the MPS GPU. The part model was trained for 6 epochs (about 1.75 minutes per epoch) and the damage model for 15 epochs; Ultralytics saves the best checkpoint by validation mask mAP after every epoch. The training script (*train_seg.py*) is generic, so either model can be retrained with more epochs or a larger backbone through command-line options.

6.4 Damage-to-Part Attribution

Figure 6.2 illustrates Algorithm 2 on a held-out photograph. The part model outlines the bumper, grille, headlight, bonnet, fender, doors and wheels; the damage model finds a broken headlight, a dent on the fender and a missing section of the front bumper. Because both models run at the original image resolution, their masks are intersected directly to find the part each damage lies on.

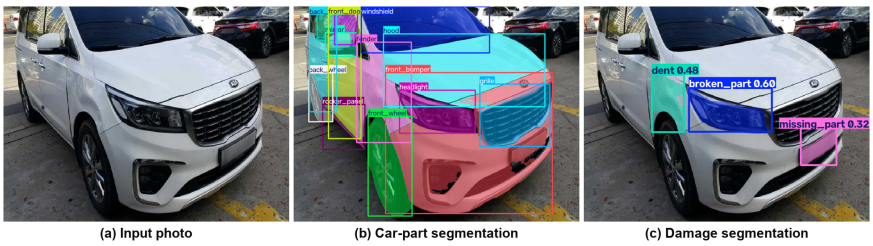


Figure 6.2: Damage-to-part attribution on a held-out photograph

6.5 Pricing Engine and Policy Rules

The pricing engine is data-driven. *pricing.json* holds a catalogue of 23 parts, each with a material, a replace price, an optional repair price, labour hours and whether it is a painted panel. Prices are stored for a hatchback and scaled by a segment multiplier (hatchback 1.0, sedan 1.35, SUV 1.8, luxury 4.0); labour is charged at ₹650, ₹500 or ₹400 per hour for tier-1, tier-2 and tier-3 cities. Table 6.4 shows an excerpt.

Table 6.4: Excerpt of the part catalogue (hatchback base prices)

Part	Material	Replace (₹)	Repair (₹)	Labour hrs (replace / repair)	Painted
Front / rear bumper	Plastic	4,500	1,500	1.5 / 2.0	Yes
Headlamp	Plastic	4,000	—	0.7	No
Bonnet	Metal	7,500	denting only	1.0 / 4.0	Yes
Front door	Metal	12,000	denting only	3.0 / 4.0	Yes
Windshield	Glass	6,500	—	1.5	No
Radiator	Metal	6,000	—	2.0	No
Tyre	Rubber	4,000	300	0.3 / 0.3	No

policy.py implements the depreciation schedule of the India Motor Tariff (rubber, nylon and plastic parts 50%, fibreglass 30%, glass nil, metal parts 0–50% by vehicle age), the compulsory deductible by engine capacity, the zero-depreciation add-on and the 75% constructive total-loss rule.

damage_rules.json maps each damage type to a repair action for the low and high plans, maps the 21 part classes to priced parts and stores a typical repair scenario for each damaged class; paint on unpainted parts such as glass is not billed.

6.6 Web Application

The user interface is built with Gradio. The left column holds the photo upload (drag-and-drop, file browsing or paste) and the vehicle and policy form: car segment, city tier, vehicle age, engine capacity, IDV, zero-depreciation add-on and voluntary deductible. The right column shows the annotated photograph and the estimate. Five sample photos allow one-click trials, and the models are cached so each request only runs inference.

6.7 Testing

The estimator is covered by 32 automated pytest tests for the depreciation schedule in every age band, deductibles, segment scaling, settlement arithmetic, zero-depreciation and total-loss cases, low-confidence flags and front/rear correction. All tests pass.

7. Results

7.1 Location and Severity Classification

Table 7.1 compares the five classifiers on the 575 held-out validation images. Transfer learning clearly beats training from scratch, and the YOLOv8n-cls model is the most accurate while also being the smallest, so it was selected for the final system.

Table 7.1: Validation accuracy of the five classifiers

Model	Validation accuracy
Baseline CNN	48.70%
CNN + Regularisation	46.61%
EfficientNet-B0 (frozen backbone)	60.70%
ResNet50 (layer4 fine-tuned)	73.91% (best epoch 77.57%)
YOLOv8n-cls (selected)	**80.35%**

Figure 7.1 shows that YOLOv8n-cls improved steadily over ten epochs, from 61.2% after the first epoch to 80.3%, with validation loss flattening near the end.

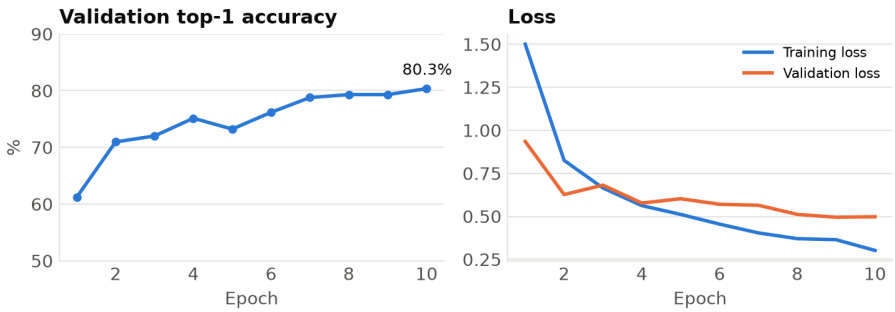


Figure 7.1: YOLOv8n-cls training: validation accuracy and loss per epoch

The per-class results (Table 7.2, Figure 7.2) show that front damage is recognised most reliably (F1 0.80–0.89), while rear crushed and rear normal photos are harder (F1 0.65–0.69), largely because those classes have fewer training images.

Table 7.2: Per-class precision, recall and F1-score of YOLOv8n-cls

Class	Precision	Recall	F1-score	Support
F_Breakage	0.88	0.90	0.89	125
F_Crushed	0.76	0.84	0.80	100
F_Normal	0.92	0.83	0.87	125
R_Breakage	0.80	0.81	0.81	75
R_Crushed	0.63	0.68	0.65	75
R_Normal	0.73	0.65	0.69	75
Weighted average	**0.81**	**0.80**	**0.80**	**575**

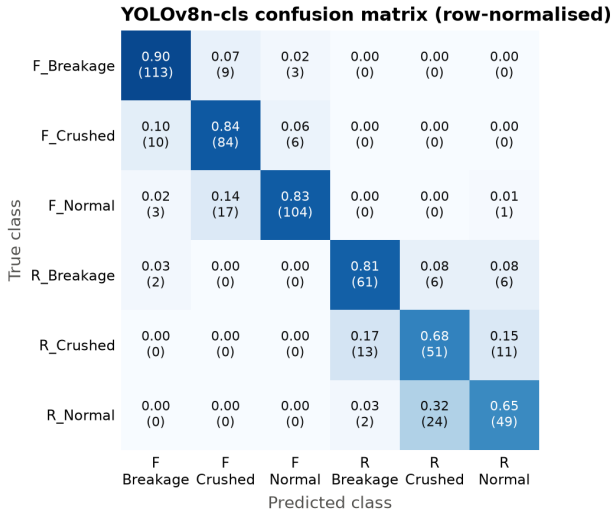


Figure 7.2: Confusion matrix of YOLOv8n-cls on the 575 validation images

7.2 Part and Damage Segmentation

Table 7.3 summarises the segmentation models. The car-part model learns quickly, reaching a mask mAP@0.5 of 0.81 after six epochs with precision 0.81 and recall 0.75.

Table 7.3: Segmentation results on the validation split

Model	Box mAP@0.5	Mask mAP@0.5	Mask mAP@0.5:0.95	Precision	Recall
Car parts (21 classes)	0.828	0.812	0.539	0.809	0.751
Damage (8 classes)	0.171	0.154	0.083	0.405	0.184

Per-class results for the part model are strong for large, distinctive parts such as wheels, windows, doors and bumpers, which are exactly the parts that dominate repair cost.

Damage segmentation is harder: damage regions are small, irregular and easily confused with reflections and dirt. The model is most reliable for the cost-driving types — cracked (0.44), missing part (0.35) and broken part (0.25) — and weakest for scratches and flaking. The classifier's severity scenario therefore defines the high estimate, so a missed scratch never hides a crushed panel, while the damage model adds part-level precision.

7.3 End-to-End Claim Estimates

Figure 7.3 and Figure 7.4 show the complete output for a held-out photograph of a sedan with front damage (vehicle age 3 years, 1,200 cc, IDV ₹6,00,000, tier-1 city). The classifier labels the photo *front, crushed*; the damage model finds a broken headlamp, a dented fender and a missing section of the front bumper. The low estimate prices exactly these three parts, ₹26,450, of which the insurer pays ₹17,564. The high estimate adds the grille, bonnet, radiator and radiator support that are typically damaged in a crushed front, giving ₹74,729 with ₹58,397 payable by the insurer.

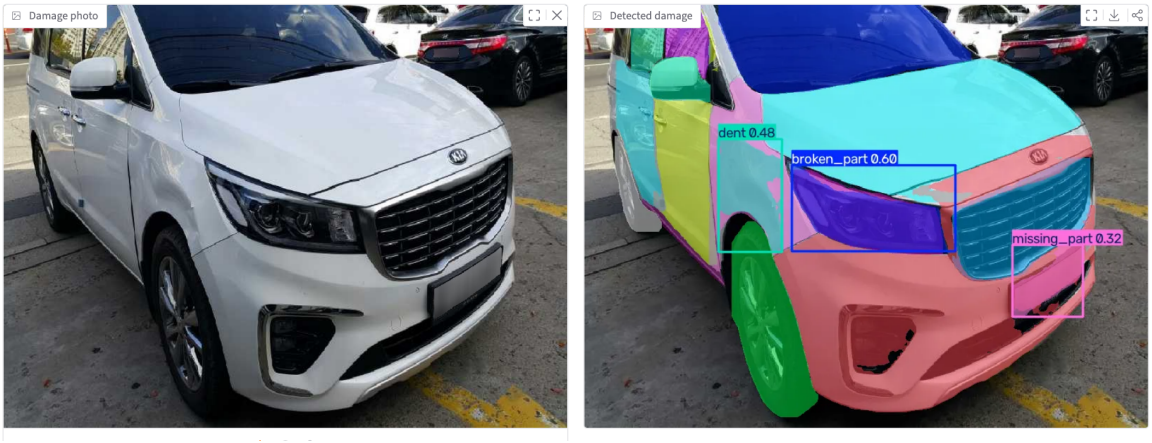
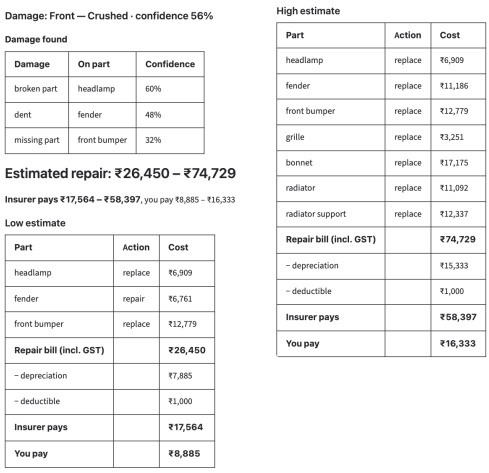


Figure 7.3: Front-damage example: uploaded photo and detections



Damage: Front — Crushed · confidence 56%

Damage found

Damage	On part	Confidence
broken part	headlamp	60%
dent	fender	48%
missing part	front bumper	32%

Estimated repair: ₹26,450 – ₹74,729

Insurer pays ₹17,564 – ₹58,397, you pay ₹8,885 – ₹16,333

Low estimate

Part	Action	Cost
headlamp	replace	₹6,909
fender	repair	₹6,761
front bumper	replace	₹12,779
Repair bill (incl. GST)		**₹26,450**
– depreciation		₹7,885
– deductible		₹1,000
Insurer pays		**₹17,564**
You pay		**₹8,885**

High estimate

Part	Action	Cost
headlamp	replace	₹6,909
fender	replace	₹11,186
front bumper	replace	₹12,779
grille	replace	₹3,251
bonnet	replace	₹17,175
radiator	replace	₹11,092
radiator support	replace	₹12,337
Repair bill (incl. GST)		**₹74,729**
– depreciation		₹15,333
– deductible		₹1,000
Insurer pays		**₹58,397**
You pay		**₹16,333**

Figure 7.4: Front-damage example: estimates and settlement

Table 7.4 traces one line item, the front bumper of this sedan, back to the rules in Chapter 4.

Table 7.4: Worked example: replacing the front bumper of a 3-year-old sedan

Component	Calculation	Amount (₹)
Part price	4,500 × 1.35 (sedan)	6,075
Labour	1.5 h × ₹650 (tier-1)	975
Paint	2,800 × 1.35	3,780
GST	18% of 10,830	1,949
Line total		**12,779**
Depreciation — part	6,075 × 1.18 × 50% (plastic)	3,584
Depreciation — paint material	3,780 × 25% × 1.18 × 50%	558

A second, rear-damage example with the zero-depreciation add-on attributes the damage to the rear bumper; the customer pays only the ₹1,000 compulsory deductible while the insurer pays ₹11,779–₹15,668.

7.4 Performance

On the Apple M4 GPU the complete pipeline — classification, both segmentation models, attribution, pricing and rendering — returns a result in about 1.9 seconds per photograph. The classifier weights occupy 2.8 MB and the segmentation models 6–28 MB, so the whole system runs comfortably on a laptop.

8. Conclusion and Future Scope

SmartClaim AI demonstrates that the first assessment of a motor insurance claim can be produced automatically, in seconds, from a single photograph and a handful of policy details. The project delivered a complete, working pipeline:

- A damage location and severity classifier chosen from five architectures, with YOLOv8n-cls reaching 80.35% validation accuracy.
- Part segmentation for 21 exterior parts (mask mAP@0.5 0.81), damage segmentation for eight damage types, and a method that attributes each damage to the part it lies on.
- A transparent pricing engine with the IRDAI depreciation, deductible, zero-depreciation and total-loss rules, presented through a web application backed by 32 automated tests.

Historical claim amounts in public datasets do not reflect visible damage, so a reliable estimator must reason about parts, prices and policy rules. Keeping the vision models and the pricing rules separate makes every estimate explainable line by line.

8.1 Future Scope

- **Larger datasets and longer training:** training the damage model on larger annotated collections, for more epochs and with bigger YOLO11 backbones to better detect scratches, paint chips and dents.
- **Multi-photo assessment and live prices:** combining several photos of a vehicle into one estimate, and replacing the static catalogue with OEM and workshop prices per make and model.
- **Fraud screening and explainability:** flagging re-used or edited images, and adding saliency maps and confidence-weighted ranges.
- **Mobile application:** guiding users to take good photos and sending the estimate to insurer claim systems.

References

[1] M. M. H. Atanasious, V. Becchetti, A. Giuseppi, A. Pietrabissa et al., “An insurtech platform to support claim management through the automatic detection and estimation of car damage from pictures,” *Electronics*, vol. 13, no. 22, Art. no. 4333, Nov. 2024, doi: 10.3390/electronics13224333.

[2] R. Khanam and M. Hussain, “YOLOv11: An overview of the key architectural enhancements,” arXiv:2410.17725, Oct. 2024.

[3] D. Lee, J. Lee and E. Park, “Automated vehicle damage classification using the three-quarter view car damage dataset and deep learning approaches,” *Heliyon*, vol. 10, no. 14, Art. no. e34016, Jul. 2024, doi: 10.1016/j.heliyon.2024.e34016.

[4] R. Varghese and S. M., “YOLOv8: A novel object detection algorithm with enhanced performance and robustness,” in *Proc. Int. Conf. on Advances in Data Engineering and Intelligent Computing Systems (ADICS)*, Chennai, India, Apr. 2024, pp. 1–6.

[5] A. S. Elbhrawy, M. A. Belal and M. S. Hassanein, “CES: Cost estimation system for enhancing the processing of car insurance claims,” *Journal of Computing and Communication*, vol. 3, no. 1, pp. 55–69, Jan. 2024.

[6] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár and R. Girshick, “Segment anything,” in *Proc. IEEE/CVF Int. Conf. on Computer Vision (ICCV)*, 2023, pp. 4015–4026.

[7] X. Wang, W. Li and Z. Wu, “CarDD: A new dataset for vision-based car damage detection,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 7, 2023, doi: 10.1109/TITS.2023.3258480.